

File di testo e binari



FILE



Esistono 2 tipi di file:

- **file di testo**
- **file binari.**

Le operazioni di I/O sono basate sul concetto di flusso (stream), ovvero una sequenza ordinata di dati che ha una sorgente e una destinazione.

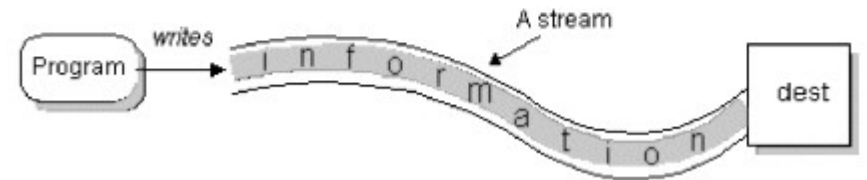
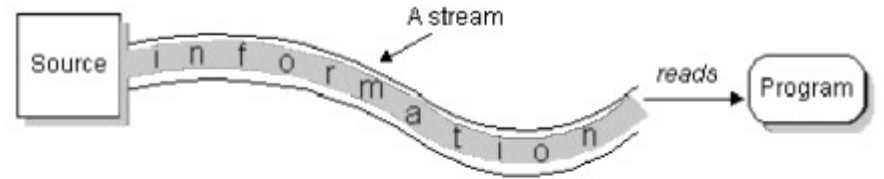
Quindi per leggere da una sorgente o per scrivere dati su una destinazione bisogna **collegarla a uno stream**, dove leggere/scrivere le informazioni in modo ordinato.

All'inizio di queste operazioni bisogna **aprire lo stream** e alla fine bisogna **chiuderlo**.

STREAM



Stream: canale di comunicazione tra un programma (Java) e una sorgente (destinazione) da cui importare (verso cui esportare) dati. L'informazione viene letta (scritta) serialmente, con modalità FIFO



CARATTERI E BYTE



Stream di caratteri:

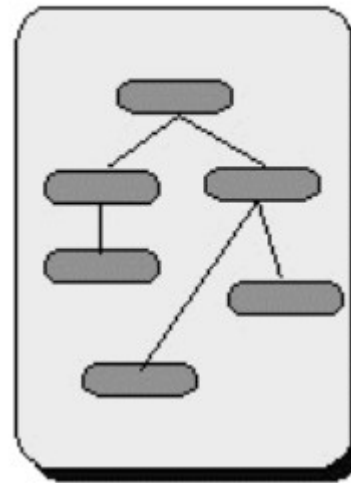
servono a leggere/scrivere

sequenze di caratteri UNICODE a 16 bit

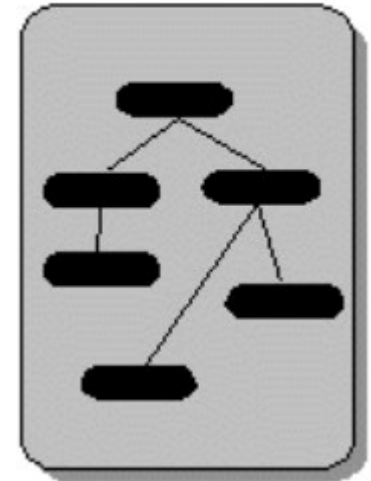
Stream di byte: servono a

leggere/scrivere sequenze di byte
(tipicamente dati)

Character Streams



Byte Streams



FILE DI TESTO E BINARI



Il package **java.io** distingue tra:

- Stream di caratteri (file di testo) →
- Stream di byte →
(file binari: immagini, video, dati)

Si traducono in famiglie di classi:

- Stream di caratteri: **Reader e Writer**
- Stream di byte **InputStream e OutputStream**

Classi per leggere/scrivere singoli caratteri/byte



- **File di testo (un carattere alla volta):**

- Lettura stream di caratteri
FileReader
- Scrittura stream di caratteri
FileWriter

- **File binari (un byte alla volta):**

- Lettura stream di byte
FileInputStream
- Scrittura stream di byte
FileOutputStream

File di testo



Leggere da un file di testo un carattere alla volta



- Passare al costruttore di **FileReader** il nome del file
- Chiamare il metodo **read()** in un ciclo fino alla fine del file
- Chiudere il file **close()**

LETTURA SU FILE DI TESTO



```
FileReader file;  
try {  
    // Lettura di tutto il contenuto del file di testo con ciclo while  
    file = new FileReader( fileName:"dati.txt");  
    // Lettura di tutti i caratteri all'interno di un file di testo  
    int next = file.read();  
    while(next != -1) {  
        char nextc = (char) next;  
        System.out.print( c:nextc); // stampa il carattere  
        next = file.read(); // legge il prossimo carattere  
    }  
    file.close();  
} catch (IOException e) {System.out.print( s:"ERRORE");}
```

Passare al costruttore di FileReader il nome del file

Chiamare il metodo read() in un ciclo fino alla fine del file

Chiudere il file close()

Scrivere su un file di testo un carattere alla volta



- Passare al costruttore di **FileWriter** il nome del file
- Chiamare il metodo **write()** in un ciclo fino alla fine del file
- Chiudere il file **close()**

SCRITTURA SU FILE DI TESTO



```
FileWriter file;  
Scanner s = new Scanner( source: System.in);  
try {  
    // Scrittura su file di testo di numeri acquisiti da tastiera  
    file = new FileWriter( fileName: "dati_output.txt");  
    String num = s.nextLine();  
    while(!num.equals( anObject: "-1")) {  
        file.write(num + " ");  
        num = s.nextLine();  
    }  
    file.close();  
} catch (IOException e) { }
```

Passare al costruttore di FileWriter il nome del file

Chiamare il metodo write() in un ciclo

Chiudere il file close()

Esercizi



- Scrivere un programma che legge un file in.txt e lo copia nel file out.txt

Input/output bufferizzato



Le classi **FileReader** e **FileWriter** forniscono i metodi di base per leggere o scrivere un carattere alla volta su file, ma leggere/scrivere un carattere alla volta richiede tempo.

Per ovviare a questo problema si possono utilizzare classi che implementano funzionalità di I/O più avanzate.

- **BufferedReader**
- **BufferedWriter**

Sono classi che usano un buffer per memorizzare temporaneamente i caratteri da leggere/scrivere in modo da ridurre il numero di accessi al file;

- **PrintWriter**

fornisce i metodi **print** e **println**, che permettono di scrivere qualunque dato Java, convertendolo automaticamente in stringa.

Esempio di lettura bufferizzata



- Per usare uno stream di input bufferizzato:
 - Passare al costruttore di **BufferedReader** uno stream basico (di `FileReader`)

Esempio:

```
BufferedReader reader = new BufferedReader(new  
FileReader("prova.txt"));
```

- Chiamare il metodo **read()** per leggere un carattere alla volta o il metodo **readLine()** per leggere una riga alla volta
- Chiudere lo stream con **close()**

Lettura buffered



```
// Lettura bufferizzata da file
```

```
FileReader fileInput;
```

```
BufferedReader bufferReader;
```

Passare al costruttore di **BufferedReader** uno stream basico (di **FileReader**)

```
try {
```

```
    fileInput = new FileReader( fileName: "prova.txt" );
```

```
    bufferReader = new BufferedReader( in: fileInput );
```

```
    String s = bufferReader.readLine();
```

```
    while (s != null) {
```

```
        System.out.println( x: s );
```

```
        s = bufferReader.readLine();
```

```
    }
```

```
    bufferReader.close();
```

```
    fileInput.close();
```

```
} catch (IOException e) {System.out.println( x: "ERRORE" );}
```

Chiamare il metodo **read()** per leggere un carattere alla volta o il metodo **readLine()** per leggere una riga alla volta

Chiudere lo stream con **close()**

Esempio di scrittura bufferizzata



- Per usare uno stream di input bufferizzato:
 - Passare al costruttore di **BufferedWriter** uno stream basico (di `FileWriter`)

Esempio:

```
BufferedWriter writer = new BufferedWriter(new  
FileWriter("prova.txt"));
```

- Chiamare il metodo **write(line)** (e metodo **flush()**) per scrivere una stringa o il metodo **newLine()** per andare a capo
- Chiudere lo stream con **close()**

Scrittura bufferizzata



Scrittura di un file con BufferWriter

```
try {  
    File file = new File("out.txt");  
    FileWriter fw = new FileWriter(file);  
    BufferedWriter bw = new BufferedWriter(fw);  
    bw.write("Prova di scrittura su file");  
    bw.flush();  
    bw.close();  
}  
catch(IOException e) { }  
}
```



```
// Scrittura buffered su file
FileWriter fileOutput;
BufferedWriter bufferWriter;
PrintWriter printWriter;
Scanner s = new Scanner( source: System.in);
try {
    // Scrittura su file testo di numeri acquisiti da tastiera
    fileOutput = new FileWriter( fileName: "dati_output2.txt");
    bufferWriter = new BufferedWriter( out: fileOutput);
    printWriter = new PrintWriter( out: bufferWriter);
    int num = s.nextInt();
    while(num != -1) {
        printWriter.println( x: num);
        num = s.nextInt();
    }
    System.out.println( x: "Dati scritti");
} catch (IOException e) {System.out.println( x: "ERRORE");}
```

Rilevare la fine del file



ATTENZIONE

Rilevare la fine del file:

- Nella lettura per righe **readLine()** fornisce **null** (la variabile deve essere di tipo stringa)
- Nella lettura carattere per carattere **read()** fornisce **-1** (la variabile deve essere di tipo int e per visualizzare il carattere letto si deve fare un'operazione di cast)

Esercizi



- Scrivere un programma che legge un file integer.txt che contiene un numero per riga e scrivere la somma a video
- Scrivere un programma che conta il numero di caratteri di un file di testo
- Scrivere un programma che legge tutte le righe di un file e le scrive in un altro file, facendo precedere ogni riga dal un numero progressivo.

Esercizio



Scrivere un programma che legga il file input.txt (il file contiene numeri, uno per riga) nei due modi:

- primo modo: leggere con la classe `BufferedReader` e scrivere il numero di caratteri del file con la classe `Buffered Writer`.
- secondo modo: leggere con `FileReader` e scrivere `FileWriter`: contare quante righe ci sono nel file.

Scrivere un programma che legga le righe del file input.txt e chiami due thread (`ThreadLineePari` e `ThreadLineeDispari`) ai quali vengono passate le righe pari nel primo e le dispari nel secondo. Chiamare l'esecuzione dei Thread e salvare, nel primo le righe ricevute in un file `pari.txt`, nel secondo le righe ricevute in un file `dispari.txt`.